

Assignment

Next-Word Prediction with Recurrent Neural Networks

Objective:

The goal of this assignment is to build, train, and evaluate a neural network capable of predicting the next word in a sentence.

Next-word prediction is a foundational task in NLP. It's the technology that powers features like autocomplete in search engines, predictive text on your phone, and is a core component of more advanced Large Language Models (LLMs). The model learns the statistical relationships between words in a large body of text to predict which word is most likely to follow a given sequence of words.

In this assignment, you will implement a model using a Long Short-Term Memory (LSTM) network, a type of Recurrent Neural Network (RNN) especially well-suited for learning from sequential data.

Core Tasks ☐ Need to follow all these tasks):

1. **Dataset:** You will use the classic novel "The Adventures of Sherlock Holmes" by Arthur Conan Doyle as your training corpus. It is large enough to learn interesting patterns but small enough to train on a standard laptop.
 - a. **Download Link:** [Project Gutenberg - The Adventures of Sherlock Holmes \(Plain Text UTF-8\)](#)
 - b. Your first task is to write a script to download and load this text file.
2. **Data Preprocessing:** Automatically process the given dataset to be used for training, including but not limited to cleaning the data, tokenization and training sequence generation.
3. **Model Implementation:** Build a neural network with an **LSTM Layer** and additional layers as required. An LSTM layer with 100-256 units is a good starting point.
 - a. You should be able to explain the model architecture.
 - b. Additionally, **implement an Attention Mechanism by adding an attention layer** on top of your LSTM network. Describe and provide reasons why you chose a specific attention mechanism.
4. **Training:** Compile the model using an appropriate loss function and an optimizer. Train the model on the sequences you generated. Monitor the loss and accuracy during training.
5. **Text Generation Function:** Create a function that uses your trained model to generate new text. This function should:

- a. Accept a "seed text" (e.g., "I saw Holmes").
- b. Tokenize and prepare the seed text as input for the model.
- c. Use the model to predict the next word.
- d. Append the predicted word to the seed text.
- e. Repeat this process in a loop to generate a passage of a desired length.
- f. For each predicted word, provide **the top 5 most likely next words (and their probabilities)**.

Python

```
def generate_text(seed_text, max_phrase_len, model):  
    """  
    Generates text by predicting the next word in a loop.  
    Also shows the top 5 predictions at each step as required.  
    """  
  
    return output_text, words_and_probabilities
```

Metrics:

1. **Training Accuracy:** A training accuracy >80%. This shows the model is learning the patterns in the text. And test accuracy > 75%.
2. **Perplexity:** This is a standard metric for evaluating language models.
 - a. Perplexity score below 250.
3. **Coherence and Readability:** The primary goal is to generate text that is readable and makes sense. Does the generated text follow basic grammatical rules? Does it tell a coherent, albeit simple, story?
4. **Submission Requirement:** In your final report, include:
 - a. 3-5 examples of generated text (at least 30 words long) from different seed phrases.
 - b. For at least one of these examples, provide a step-by-step breakdown showing the top 5 most likely next words (and their probabilities) that the model considered at each step of the generation. This demonstrates an understanding of the model's decision process.

Evaluation Criteria:

1. **Functionality:** Does the solution implement all the tasks?
2. **Code Quality:** Is the code well-organized, documented, and follows best practices?
3. **Innovation:** Are there any additional features or improvements beyond the basic tasks?
4. **Output:** Is the output understandable?

- a. **A Good Output:** the core concept involves training on vast datasets to develop generative models
- b. **A Bad Output:** the core concept involves training training on vast datasets to to to generative models models models and models or

Submission

Please submit a link to a **GitHub** repository containing:

- A Jupyter Notebook or Python script with your fully documented code.
- Using **poetry** or **uv** to manage dependencies. Do not use requirements.txt.
- A README.md file explaining how to run your code.
- A brief report summarizing your approach, final metrics, and the examples of generated text.

Notes:

- Do not limit yourself.
- For final evaluation, we may use a different seed phrase.
- Show us the architecture that you have followed.

Good luck!

If you have any questions or need any clarification regarding the assignment, please feel free to reach out to hr@abottstech.com, and we will get back to you accordingly